Fine-tuning

Huấn luyện tiếp model có sẵn.

> Bộ tìm kiếm của câu lạc bộ đã có baseline dùng embedding cố định. Tuy nhiên, ảnh bảng viết tay và ảnh hoạt động trong lớp khác dữ liệu mà encoder từng học. Một head nhỏ có thể chưa đọc ra nhãn tốt. Ta có nên huấn luyện lại toàn bộ mạng ngay không? Fine-tuning cho phép điều chỉnh trọng số đã học cho nhiệm vụ mới, nhưng phải chọn phần cần mở, learning rate và checkpoint bằng validation. Bài này đi từ head mới trên encoder đóng băng đến mở encoder, rồi kiểm cả tham số lẫn buffer để biết mô hình thực sự đã thay đổi ở đâu.

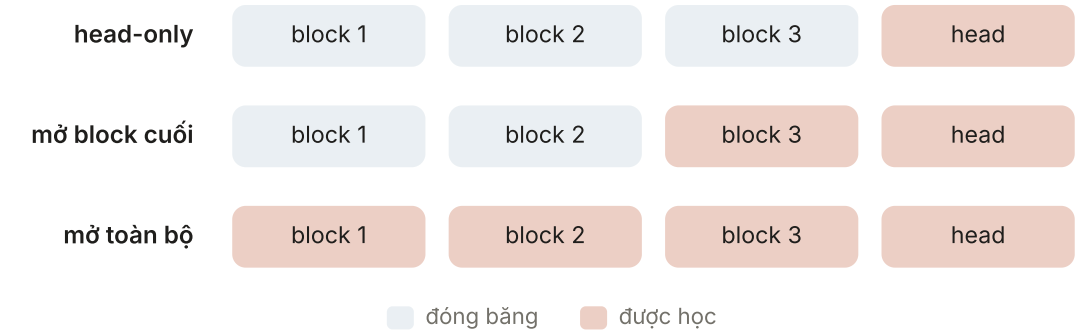
MỤC TIÊU SAU BÀI NÀY

- Thay head nhãn cũ bằng head của bài mới, rồi đóng băng hoặc mở encoder có chủ đích.
- Phân biệt `requires_grad`, `train()/eval()`, `no_grad()` và việc optimizer cập nhật.
- Dùng learning rate riêng cho encoder và head, chọn checkpoint theo validation.
- Phát hiện BatchNorm vẫn đổi buffer dù trọng số đã đóng băng.

Chuyển kiến thức nào sang nhiệm vụ mới?

Model pretrained gồm encoder và thường có một head phục vụ nhiệm vụ cũ. Encoder ảnh có thể học nét, cấu trúc và những tổ hợp hình dạng có ích cho nhiều nhãn khác. Nhưng head phân loại nhãn cũ không có lý do biết các nhãn mới của câu lạc bộ. Ta giữ encoder và thay head có đúng số lớp K của bài mới.

Viết $\mathbf{z} = f_\theta(\mathbf{x})$ và logits $\boldsymbol{\ell} = g_\phi(\mathbf{z})$. θ là tham số encoder, ϕ là tham số head. Head-only training chỉ cập nhật ϕ. Fine-tuning mở một phần hoặc toàn bộ θ cùng với ϕ. Phần mở nhiều hơn có thể thích nghi tốt hơn nhưng cũng có thể học thuộc dữ liệu nhỏ hoặc làm mất biểu diễn có ích.



Ba mức mở tham số, thử theo thứ tự: chỉ head, thêm block cuối, rồi toàn bộ encoder.



bước 0: cả hai head mới đều ngẫu nhiên

bước 10: accuracy validation (400 ảnh) từ ImageNet 0.80, từ ngẫu nhiên 0.54

bước 50: accuracy validation (400 ảnh) từ ImageNet 0.86, từ ngẫu nhiên 0.59

bước 150: accuracy validation (400 ảnh) từ ImageNet 0.88, từ ngẫu nhiên 0.68

ResNet-18 học bốn lớp CIFAR-10 từ 800 ảnh, cùng số bước: bắt đầu từ trọng số ImageNet đoán đúng sớm hơn hẳn bắt đầu từ ngẫu nhiên.

HỎI NHANH

Nhiệm vụ cũ có 1000 lớp, bài mới có 3 lớp. Nạp checkpoint xong, phần nào chắc chắn phải thay hoặc kiểm trước khi train?

A Batch size, vì đầu ra 3 lớp cần batch chia hết cho 3

B Head cuối, và bảng đổi ba nhãn mới sang chỉ số 0, 1, 2

C Seed, vì head cũ được khởi tạo bằng một seed khác

D Learning rate của encoder, vì 1000 lớp cần bước nhỏ hơn

Trong mạng lớn, các block gần input thường bắt dấu hiệu chung như nét, cạnh, mảng màu; các block sau mang đặc trưng sát nhiệm vụ cũ hơn. Vì vậy thứ tự thử thực dụng là head trước, rồi mở thêm block cuối, rồi mới đến toàn bộ, đúng ba hàng của hình trên. Mở dần từng phần như vậy (gradual unfreezing) cũng là cách ULMFiT fine-tune model

ngôn ngữ. Khi miền đích khác hẳn nguồn, như ảnh X-quang so với ảnh đời thường, đặc trưng ở các block đầu cũng có thể cần đổi; mỗi mức mở khi đó là một thí nghiệm, và baseline head-only là mốc để biết thay đổi có đáng giá.

HỎI NHANH

Dữ liệu đích ít và khá giống miền nguồn, head-only đã có điểm validation. Ngân sách chỉ đủ thử thêm một cấu hình. Nên thử gì?

A Mở toàn bộ mạng, với cùng learning rate của head

B Khởi tạo lại encoder rồi học từ đầu trên dữ liệu đích

C Mở block đầu gần input, vì đặc trưng nền quan trọng nhất

D Mở thêm block cuối, giữ các block đầu đóng băng

Một mô hình nguồn tự pretrain

Để thấy mọi bước mà không cần tải trọng số, ta tự tạo một mô hình nguồn ngay trong notebook từ bộ chữ số 8×8 của scikit-learn. Nhiệm vụ nguồn phân loại các chữ số 0–7; nhiệm vụ đích phân biệt 8 và 9. Mọi ảnh 8 hoặc 9 đều bị loại khỏi việc học nguồn, nên encoder nguồn đóng đúng vai một checkpoint pretrained: đã học đặc trưng hữu ích trên dữ liệu khác và chưa từng thấy nhãn đích. Với ResNet hay DistilBERT, các bước phía sau giống hệt; chỉ khác ở chỗ trọng số được nạp từ tệp checkpoint thay vì vừa học xong.

Một encoder `64 → 32 → ReLU` đủ để thực nghiệm chạy trong vài giây. Pixel của bộ dữ liệu này nằm từ 0 đến 16, nên input được chia 16; mọi bước sau, kể cả nhiệm vụ đích, phải dùng đúng phép chia ấy, giống như fine-tune một checkpoint thật phải dùng đúng processor của nó (bài 4.1). Một phần ảnh nguồn được giữ làm validation để kiểm encoder thực sự đã học, thay vì gọi trọng số ngẫu nhiên là "đã học".

```
Python
```

```
import copy
import numpy as np
import torch
from torch import nn
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split

torch.set_num_threads(1)
torch.manual_seed(3)
digits = load_digits()
source_ids = np.flatnonzero(digits.target < 8)
target_ids = np.flatnonzero(digits.target >= 8)
source_train, source_valid = train_test_split(
    source_ids, test_size=0.25, stratify=digits.target[source_ids], random_state=3
)

def inputs(ids):
    return torch.tensor(digits.data[ids] / 16.0, dtype=torch.float32)

source_encoder = nn.Sequential(nn.Linear(64, 32), nn.ReLU())
source_head = nn.Linear(32, 8)
source_model = nn.Sequential(source_encoder, source_head)
source_optimizer = torch.optim.Adam(source_model.parameters(), lr=0.01)
criterion = nn.CrossEntropyLoss()
source_x = inputs(source_train)
source_y = torch.tensor(digits.target[source_train], dtype=torch.long)
for _ in range(80):
    source_model.train()
    source_optimizer.zero_grad(set_to_none=True)
    loss = criterion(source_model(source_x), source_y)
    loss.backward()
    source_optimizer.step()

source_model.eval()
with torch.no_grad():
    source_prediction = source_model(inputs(source_valid)).argmax(dim=1).numpy()
print("source train/valid:", len(source_train), len(source_valid))
print("source valid accuracy:", round(float((source_prediction ==
digits.target[source_valid]).mean()), 4))
print("source gap target:", len(set(source_ids) & set(target_ids)) == 0)
```

KẾT QUẢ

```
source train/valid: 1082 361
source valid accuracy: 0.9806
source gap target: True
```

Accuracy 0.9806 trên 361 ảnh nguồn giữ lại cho thấy encoder đã học được đặc trưng chữ số có ích. Dòng thứ ba xác nhận không ảnh 8 hay 9 nào lọt vào việc học nguồn; nếu có, điểm của nhiệm vụ đích sau này sẽ đo cả phần encoder đã thuộc lòng, không chỉ phần kiến thức được chuyển. Split ở đây chia theo từng ảnh; với dữ liệu có nhiều ảnh của cùng một người hay cùng một buổi, split phải chia theo nhóm ấy (bài 1.3).

HỎI NHANH

Một đội pretrain encoder trên toàn bộ 1797 ảnh chữ số từ 0 đến 9 cùng nhãn của chúng, rồi báo accuracy của nhiệm vụ phân biệt 8 với 9 trên validation lấy từ chính bộ ảnh ấy. Con số này đo gì?

A Đúng khả năng chuyển học, vì head đích vẫn là head mới

B Đúng khả năng chuyển học, vì phần nguồn không dùng head đích

C Một con số quá lạc quan, vì encoder đã học cả ảnh validation

D Một con số quá bi quan, vì nhãn 0 đến 7 làm nhiễu encoder

Bắt đầu bằng một head mới

Ta dành 48 ảnh đích làm train có nhãn; các ảnh đích còn lại làm validation. Nhãn 8 đổi thành 0, nhãn 9 đổi thành 1 để khớp head hai lớp. Việc đổi nhãn phải lưu lại để trả output đúng tên lớp sau khi dự đoán.

Python

```
target_train, target_valid = train_test_split(
    target_ids, train_size=48, random_state=5, stratify=digits.target[target_ids]
)
target_x = inputs(target_train)
target_y = torch.tensor(digits.target[target_train] - 8, dtype=torch.long)
valid_x = inputs(target_valid)
valid_y = torch.tensor(digits.target[target_valid] - 8, dtype=torch.long)
```

```
torch.manual_seed(10)
initial_head = nn.Linear(32, 2)
head_only = nn.Sequential(copy.deepcopy(source_encoder), copy.deepcopy(initial_head))
for parameter in head_only[0].parameters():
    parameter.requires_grad_(False)
optimizer = torch.optim.SGD(head_only[1].parameters(), lr=0.05)
before = {name: value.detach().clone() for name, value in head_only[0].named_parameters()}

head_only.train()
head_only[0].eval()
optimizer.zero_grad(set_to_none=True)
loss = criterion(head_only(target_x), target_y)
loss.backward()
print("encoder grad None:", all(p.grad is None for p in head_only[0].parameters()))
print("head co gradient:", head_only[1].weight.grad.norm().item() > 0)
optimizer.step()
print("encoder khong doi:", all(torch.equal(before[n], p) for n, p in
head_only[0].named_parameters()))
print("target train/valid:", len(target_train), len(target_valid))
```

KẾT QUẢ

```
encoder grad None: True
head co gradient: True
encoder khong doi: True
target train/valid: 48 306
```

Ở mạng này, encoder chỉ có Linear và ReLU; `eval()` chưa đổi phép tính của nó. Ta vẫn gọi để tạo thói quen đúng khi sau này dùng encoder có Dropout hoặc BatchNorm. Dòng `encoder grad None` kiểm việc không tạo gradient cho tham số đã đóng băng. Dòng thứ ba so từng giá trị tensor trước và sau một bước, kiểm việc optimizer không sửa encoder. Nếu chỉ nhìn loss giảm, ta không biết phần nào đã được cập nhật.

HỎI NHANH

Một tham số có `requires_grad=False` nhưng vẫn nằm trong forward. Khi dự đoán, nó đóng vai trò gì?

A Vẫn tính forward như cũ, chỉ không được học

B Bị bỏ qua, nên output của tầng ấy bằng 0

C Bị thay bằng giá trị trung bình của nó

D Bị tách khỏi mạng cho tới khi gọi lại `train()`

HỎI NHANH

Head hai lớp được học với nhãn `digit - 8`. Validation accuracy đạt 0.94, nhưng file nộp ghi thẳng `argmax` của logits và được chấm 0 điểm. Lỗi khả dĩ nhất là gì?

A Head hai lớp không đủ sức cho dữ liệu test

B Validation bị rò, nên 0.94 là con số ảo

C File nộp ghi chỉ số 0, 1 thay vì 8, 9

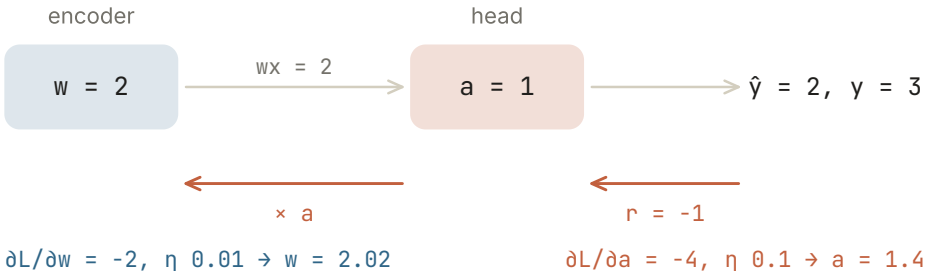
D Encoder ở chế độ `eval()` khi dự đoán test

Ví dụ tính tay: vì sao encoder thường dùng bước nhỏ hơn?

Xét mô hình một chiều $\hat{y} = a(wx)$, encoder có tham số w, head có tham số a, loss $L = (\hat{y} - y)^2$. Với $x = 1$, $w = 2$, $a = 1$, $y = 3$, ta có $\hat{y} = 2$, phần dư -1, nên

$$\frac{\partial L}{\partial a} = 2(-1) \cdot 2 = -4, \qquad \frac{\partial L}{\partial w} = 2(-1) \cdot 1 = -2.$$

Nếu learning rate head là 0.1, encoder là 0.01, cập nhật cho $a = 1.4$ và $w = 2.02$. Nếu encoder đóng băng thì w vẫn bằng 2.



Ví dụ tính tay ŷ = a(wx): gradient về encoder phải nhân qua a của head; với learning rate 0.1 cho head và 0.01 cho encoder, a lên 1.4 còn w chỉ nhích tới 2.02.

Python

```
x, w, a, y = 1.0, 2.0, 1.0, 3.0
residual = a * w * x - y
grad_a = 2 * residual * w * x
grad_w = 2 * residual * a * x
print("gradient:", grad_a, grad_w)
print("sau buoc:", a - 0.1 * grad_a, w - 0.01 * grad_w)
```

KẾT QUẢ

```
gradient: -4.0 -2.0
sau buoc: 1.4 2.02
```

Để ý $\partial L/\partial w$ chứa a: gradient về encoder phải đi qua head. Head mới khởi tạo ngẫu nhiên, nên trong những bước đầu, hướng gradient mà encoder nhận được cũng mang theo sự ngẫu nhiên đó. Nếu encoder đi bước lớn như head, nó có thể bị kéo xa chỉ để bù cho một head còn sai, làm hỏng những đặc trưng tốt đã học. Việc mô hình mất dần kiến thức nguồn khi được huấn luyện tiếp cho nhiệm vụ mới gọi là quên thảm khốc (catastrophic forgetting); bước lớn ở phần pretrained làm hiện tượng này đến nhanh hơn. Learning rate nhỏ cho encoder, hoặc học head-only vài epoch rồi mới mở encoder, hạn chế tác hại ấy. Tỉ lệ 10 lần ở ví dụ là một lựa chọn, không phải hằng số. Với BERT, bài báo gốc fine-tune toàn bộ mô hình với learning rate từ $2 \cdot 10^{-5}$ đến $5 \cdot 10^{-5}$ trong 2–4 epoch, nhỏ hơn mức 10^{-4} dùng khi pretrain chính BERT.

HỎI NHANH

Vẫn ví dụ ấy, với $x = 1$, $w = 2$, $y = 3$, nhưng head mới được khởi tạo $a = -1$. Gradient theo w bằng bao nhiêu, và gradient descent đẩy w theo hướng nào?

A -2, nên w tăng, như ví dụ có $a = 1$

B -10, nên w tăng để $\hat{y}$ lớn lên

C 0, nên w đứng yên chờ head học xong

D 10, nên w giảm

So sánh head-only và mở encoder

Hai thực nghiệm bắt đầu từ cùng encoder nguồn và cùng head mới, nên khác biệt chỉ nằm ở việc mở encoder. Ta chạy 60 epoch full-batch, chấm validation sau mỗi epoch và giữ checkpoint có validation loss thấp nhất. Head dùng learning rate 0.05; khi mở, encoder có nhóm riêng với 0.003, nhỏ hơn khoảng 17 lần. Vì validation vừa dùng để chọn epoch vừa để so hai mode, con số báo cáo cuối cùng nên lấy từ một test giữ kín.

Python

```
def run_transfer(open_encoder):
    transferred = nn.Sequential(copy.deepcopy(source_encoder), copy.deepcopy(initial_head))
    for parameter in transferred[0].parameters():
        parameter.requires_grad_(open_encoder)
    groups = [{"params": transferred[1].parameters(), "lr": 0.05}]
    if open_encoder:
        groups.append({"params": transferred[0].parameters(), "lr": 0.003})
    optimizer = torch.optim.Adam(groups)
    best_loss = float("inf")
    best_state, best_epoch = None, None
    history = []
    for epoch in range(1, 61):
        transferred.train()
        if not open_encoder:
            transferred[0].eval()
```

```
            optimizer.zero_grad(set_to_none=True)
            train_loss = criterion(transferred(target_x), target_y)
            train_loss.backward()
            optimizer.step()
            transferred.eval()
            with torch.no_grad():
                val_loss = criterion(transferred(valid_x), valid_y).item()
            history.append((train_loss.item(), val_loss))
            if val_loss < best_loss:
                best_loss = val_loss
                best_epoch = epoch
                best_state = copy.deepcopy(transferred.state_dict())
        transferred.load_state_dict(best_state)
        with torch.no_grad():
            accuracy = (transferred(valid_x).argmax(dim=1) == valid_y).float().mean().item()
        trainable = sum(p.numel() for p in transferred.parameters() if p.requires_grad)
        return transferred, history, best_epoch, best_loss, accuracy, trainable

    for opened in [False, True]:
        net, history, epoch, val_loss, accuracy, count = run_transfer(opened)
        print("open encoder =", opened, "| trainable =", count, "| best epoch =", epoch)
        print("best val loss =", round(val_loss, 4),
              "| epoch 60 val loss =", round(history[-1][1], 4),
              "| val accuracy =", round(accuracy, 4))
```

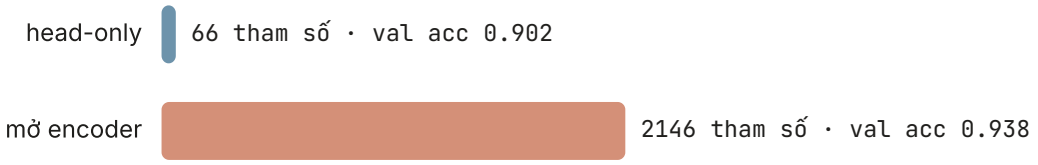
KẾT QUẢ

```
open encoder = False | trainable = 66 | best epoch = 36
best val loss = 0.2247 | epoch 60 val loss = 0.2366 | val accuracy = 0.902
open encoder = True | trainable = 2146 | best epoch = 60
best val loss = 0.157 | epoch 60 val loss = 0.157 | val accuracy = 0.9379
```

Đọc loss và accuracy ở checkpoint được chọn, không chỉ ở epoch cuối. Head-only học 66 tham số ($32 \cdot 2 + 2$); validation loss thấp nhất 0.2247 ở epoch 36, đến epoch 60 là 0.2366 trong khi train loss vẫn giảm. Giữ checkpoint epoch 36 nghĩa là không phải trả giá cho 24 epoch học thêm vô ích. Mở encoder học 2146 tham số (thêm $64 \cdot 32 + 32 = 2080$ của encoder), loss 0.1570 và accuracy 0.9379, so với 0.9020 của head-only: đúng thêm 11 trong 306 ảnh validation. Checkpoint tốt nhất của nó rơi đúng epoch cuối, tức validation còn đang giảm; ngân sách 60 epoch có thể chưa đủ, và đó là tín hiệu để chạy dài hơn, không phải bằng chứng đã hội tụ.

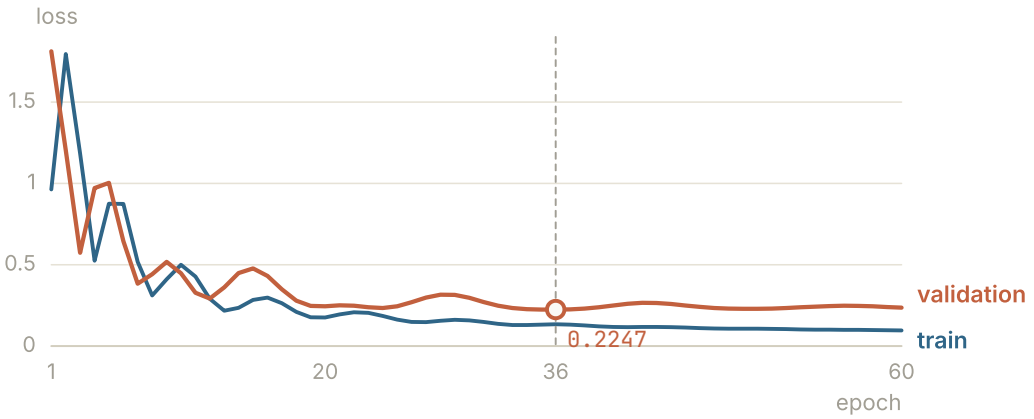
Trên một split nhỏ, chênh lệch 11 ảnh là bằng chứng có ích, chưa phải định luật "mở encoder luôn thắng". Loss và accuracy cũng có thể chọn hai epoch khác nhau: cross-

entropy còn phạt sự tự tin sai, còn accuracy chỉ nhìn argmax. Ở đây encoder chỉ có một tầng Linear nên mở nó là mở toàn bộ; với ResNet, "mở block cuối" là một tập tham số riêng, có số lượng và bộ nhớ riêng phải đo lại. Mỗi tham số được mở kéo theo một gradient và, với Adam, hai moment m, v: thêm ba số float32 cho mỗi tham số, chưa kể activation phải giữ để backward qua phần mở (bài 3.7). Với 66 tham số, phần này không đáng kể; với hàng chục triệu tham số, nó lên tới hàng trăm MB.

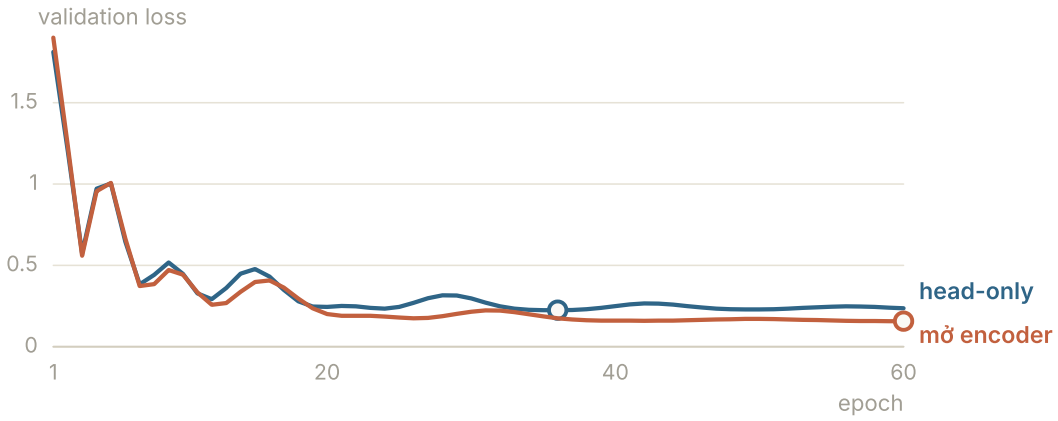


đúng thêm 11 trên 306 ảnh validation; mỗi tham số mở kèm gradient và hai moment Adam

Số tham số được học và accuracy validation ở checkpoint (số từ code bài): mở encoder tăng từ 66 lên 2146 tham số, accuracy từ 0.902 lên 0.938.



Head-only (số từ code bài): train loss giảm tới epoch 60, validation thấp nhất 0.2247 ở epoch 36 rồi nhích lên, nên checkpoint giữ epoch 36.



Validation loss của hai cách (số từ code bài, chấm tròn là checkpoint): mở encoder với learning rate nhỏ xuống 0.157 và còn đang giảm ở epoch 60; head-only dừng ở 0.2247.

HỎI NHANH

Một lần fine-tune 20 epoch có checkpoint tốt nhất ở epoch 20, và validation loss vẫn đang giảm. Hiểu kết quả này thế nào?

A Có thể chưa đủ epoch, nên chạy thêm rồi xem tiếp

B Đã hội tụ đúng ở epoch 20, nên dừng tại đó

C Đã overfit, vì điểm tốt nhất rơi vào epoch cuối

D Chọn lại theo train loss, vì validation chưa ổn

Recipe của chương đưa đúng thí nghiệm này lên DistilBERT với asset và dữ liệu local. Mode `head` đóng băng backbone và giữ nó ở eval; mode `full` học backbone với learning rate 10^{-5} và head với 10^{-3}. Cả hai chọn checkpoint theo macro-F1 validation và ghi số tham số học, lịch sử validation, thời gian vào `RESULT.json`; so hai tệp ấy giống như so hai cặp dòng output trên.

Freeze, eval và BatchNorm là ba chuyện

BatchNorm có tham số affine như weight và bias, đồng thời có buffer `running_mean`, `running_var`. Trong train mode, batch mới có thể cập nhật buffer dù không tính gradient. Với mặc định theo dõi running statistics, chuyển sang eval dùng buffer đã tích lũy. Quyết định có cập nhật thống kê miền mới hay không là một phần của fine-tuning; với encoder cần giữ cố định, ta giữ cả buffer bằng eval. Khi batch fine-tune nhỏ, như vài ảnh độ phân giải cao mỗi batch, thống kê batch rất nhiễu, nên nhiều hệ thống phát hiện vật thể giữ BatchNorm của backbone cố định ngay cả khi các trọng số khác được học.

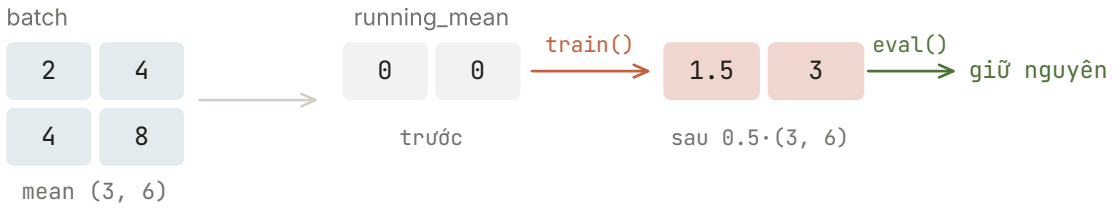
Thí nghiệm sau đặt mọi tham số BatchNorm thành không học, nhưng vẫn cho một batch qua train mode. Trung bình batch là $(3, 6)$, `momentum=0.5`, trung bình cũ là $(0, 0)$, nên trung bình mới là $(1.5, 3)$. Trong BatchNorm, momentum này là hệ số cập nhật thống kê, khác nghĩa với momentum của optimizer.

Python

```python
bn = nn.BatchNorm1d(2, momentum=0.5)
for parameter in bn.parameters():
    parameter.requires_grad_(False)
batch = torch.tensor([[2., 4.], [4., 8.]])
bn.train()
with torch.no_grad():
    bn(batch)
after_train = bn.running_mean.clone()
bn.eval()
with torch.no_grad():
    bn(batch + 100)
print("running mean sau train:", after_train.tolist())
print("eval giu buffer:", torch.equal(after_train, bn.running_mean))
print("khop tinh tay:", torch.allclose(after_train, batch.mean(dim=0) * 0.5))
```

KẾT QUẢ

```
running mean sau train: [1.5, 3.0]
eval giu buffer: True
khop tinh tay: True
```



mọi tham số đã requires_grad=False, đang trong no_grad(), mà buffer vẫn đổi

BatchNorm đã đóng băng hết tham số vẫn cập nhật running_mean khi chạy ở train(): từ (0, 0) thành (1.5, 3); chỉ eval() giữ buffer đứng yên.

Dòng đầu trực tiếp phản bác hiểu lầm "không có gradient thì không có gì thay đổi". Dropout lại không có running mean; eval làm nó bỏ thao tác che ngẫu nhiên. Khi gọi `model.train()` ở đầu epoch, các module con cũng chuyển train; nếu encoder phải đứng yên, gọi `encoder.eval()` sau đó. Còn `no_grad()` không được bọc phần đang học (head, adapter): autograd phải ghi lại các phép tính ấy thì backward mới tới được tham số của chúng.

HỎI NHANH

Đóng băng weight và bias của BatchNorm, rồi cho một batch mới đi qua ở chế độ `train()`, bên trong `no_grad()`. Running mean có đổi không?

A Không, vì mọi tham số đều có `requires_grad=False`

B Không, vì `no_grad()` chặn mọi thay đổi trong forward

C Có, vì buffer tự cập nhật ngay trong forward

D Có, nhưng chỉ khi gọi `optimizer.step()` sau đó

HỎI NHANH

`model = nn.Sequential(encoder, head)`, và encoder có BatchNorm cần giữ nguyên. Đoạn sau có giữ được running mean của encoder không?

```
encoder.eval()            # gọi một lần trước vòng lặp
for epoch in range(10):
    model.train()
    for x, y in loader:
        ...               # forward, backward, step cho head
```

A Có, vì `encoder.eval()` đã được gọi từ trước

B Không, vì `model.train()` lan xuống cả encoder

C Có, nếu tham số của encoder đã `requires_grad=False`

D Không, vì optimizer cập nhật buffer ở mỗi `step()`

Bẫy cài đặt

Optimizer chỉ giữ danh sách tham số có lúc nó được tạo. Sau khi mở thêm block, optimizer cũ có thể vẫn chỉ chứa head; ngược lại, nếu tạo optimizer rồi mới thay head, head mới có gradient sau `backward()` nhưng không nằm trong nhóm nào, nên `step()` để nó nguyên như lúc khởi tạo. Kiểm `param_groups` hoặc dựng optimizer mới cho đúng tập tham số; khi đổi optimizer cần hiểu trạng thái cũ có được giữ hay không. Lưu checkpoint tốt nhất bằng copy thật hoặc ghi tệp ngay; `best_state = model.state_dict()` giữ tham chiếu tensor, nên các epoch sau có thể làm "checkpoint tốt nhất" đổi theo.

Khi so hai cách fine-tune, chỉ đổi đúng một thứ. Head-only dùng processor này, full fine-tuning dùng processor khác thì chênh lệch điểm không còn quy được cho việc mở encoder. Giữ cùng split, metric, dữ liệu và seed; thời gian và bộ nhớ ghi riêng, đo trên đúng môi trường sẽ chạy. Với ít nhãn, augmentation, regularization và early stopping giúp giảm học thuộc, nhưng augmentation phải giữ nguyên nhãn: xoay 180° biến chữ 9 thành một hình giống 6, nên phép xoay ấy không hợp lệ cho bài phân biệt 8 và 9.

HỎI NHANH

Head-only dùng ảnh 224×224 chuẩn hóa theo mean và std của checkpoint, đạt 0.81. Full fine-tuning dùng ảnh 160×160 không chuẩn hóa, đạt 0.78, trên cùng validation. Kết luận nào có cơ sở?

A Mở encoder làm hại, nên dừng hẳn ở head-only

B Ảnh 160×160 kém hơn, nên luôn dùng ảnh 224×224

C Full fine-tuning thắng, nếu tính theo thời gian chạy

D Chưa kết luận được, phải chạy lại với cùng processor

Khi đi thi

- **Baseline head-only trước.** Vài phút học head trên encoder đóng băng cho một mốc điểm và một submission hợp lệ; mọi mức mở thêm phải thắng mốc này trên cùng validation.
- **Chạy thử một batch trước khi chạy dài.** In số tham số học, xem `param_groups`, kiểm phần mở có gradient và đo bộ nhớ GPU ở batch thật. Optimizer tạo trước khi thay head, hay block mở mà không có trong optimizer, đều lộ ra ở bước này.
- **Learning rate nhỏ cho phần pretrained.** Model cỡ BERT thường fine-tune với learning rate từ $2 \cdot 10^{-5}$ đến $5 \cdot 10^{-5}$ trong 2–4 epoch. Validation loss tăng ngay sau epoch đầu là dấu hiệu bước quá lớn hoặc học thuộc: giảm learning rate, mở ít hơn, giữ checkpoint tốt nhất.
- **Đúng tiền xử lý của checkpoint.** Kích thước ảnh, mean/std, tokenizer phải là của checkpoint; đổi processor giữa hai lần chạy là so hai thí nghiệm khác nhau.
- **Kiểm ánh xạ nhãn và file nộp.** Head trả chỉ số $0, \ldots, K-1$; đổi lại tên lớp gốc trước khi ghi, và dự đoán toàn bộ test bằng đúng checkpoint đã chọn, ở `eval()`.

- **Ngân sách theo môi trường thi.** GPU, thời gian và việc có sẵn checkpoint hay không lấy theo thông báo của cuộc thi; đo thời gian một epoch trên máy thi trước khi quyết định mở bao nhiêu.

Tóm tắt

Fine-tuning đưa kiến thức của mô hình nguồn sang nhiệm vụ đích. Head mới trên encoder đóng băng là mốc đầu; mở thêm encoder có chủ đích, với learning rate nhỏ hơn head, và chọn checkpoint bằng validation. Freeze tham số, chế độ train/eval, `no_grad()` và tập tham số của optimizer là bốn công tắc khác nhau; BatchNorm còn có buffer đổi ngay trong forward. Trên ví dụ chữ số, mở encoder nâng validation accuracy từ 0.9020 lên 0.9379; với model lớn, cùng quy trình ấy được chạy và đo lại trên dữ liệu của bài.

Kiểm tra cuối bài

CÂU 1

Một đội có 3000 bình luận tiếng Anh gán ba nhãn cảm xúc, và một GPU dùng trong hai giờ. Nên xuất phát từ đâu?

A BERT pretrained, thay head ba lớp rồi fine-tune nhẹ

B Transformer cỡ BERT khởi tạo ngẫu nhiên, học từ đầu trên 3000 câu

C ResNet-50 pretrained trên ImageNet, thay head ba lớp rồi fine-tune

D AlexNet học từ đầu trên vector đếm từ của 3000 bình luận

PHỎNG THEO NOAI 2025 Singapore · Phần 1 · Câu 17

CÂU 2

Ta muốn giữ cố định encoder khi học head, đồng thời tiết kiệm bộ nhớ và thời gian của backward. Cách nào đạt được cả hai?

A Đặt learning rate 0 cho nhóm tham số của encoder

B Gọi `requires_grad_(False)` cho các tham số của encoder

C Gọi `encoder.eval()` trước mỗi epoch train

D Gọi `model.freeze()` một lần trước vòng lặp train

PHỎNG THEO NOAI 2026 Singapore · Vòng sơ loại · Câu 132

CÂU 3

Ba đội cùng dùng một BERT pretrained cho bài phân loại. Đội A viết prompt kèm 5 ví dụ có nhãn. Đội B đóng băng BERT, học một head mới trên vector `[CLS]`. Đội C cập nhật cả trọng số BERT lẫn head trên 2000 câu có nhãn. Đội nào đã làm thay đổi trọng số pretrained?

A Cả ba đội, vì đều dùng dữ liệu có nhãn của bài

B Chỉ đội C

C Đội B và đội C, vì cả hai đều huấn luyện

D Đội A và đội C, vì cả hai đều đưa ví dụ vào mô hình

PHỎNG THEO NOAI 2026 Singapore · Vòng sơ loại · Câu 245

CÂU 4

Fine-tune ResNet-50 pretrained cho 10 lớp:

```
model = resnet50(weights="IMAGENET1K_V2")
for p in model.parameters():
    p.requires_grad = False
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)
model.fc = nn.Linear(2048, 10)
# vòng train: zero_grad, forward, loss.backward(), optimizer.step()
```

Sau khi train, validation accuracy chỉ khoảng 10%, và `model.fc.weight` giống hệt lúc khởi tạo. Lỗi ở đâu?

A Optimizer được tạo trước khi thay `fc`, nên không chứa `fc` mới

B Backbone đóng băng chặn gradient, không cho đi tới `fc`

C Learning rate 1e-3 quá nhỏ cho một head khởi tạo ngẫu nhiên

D Optimizer nhận tham số đóng băng nên bỏ qua cả nhóm

PHỎNG THEO NOAI 2026 Singapore · Chung kết · Câu 15

CÂU 5

Fine-tune BERT-base phân loại 5 lớp trên vài nghìn câu: batch 32, `max_length=128`, AdamW với learning rate 10^{-3} cho toàn bộ mô hình, 3 epoch. Validation loss giảm sau epoch 1 rồi tăng mạnh, còn train loss vẫn giảm đều. Nguyên nhân khả dĩ nhất là gì?

A Batch 32 quá nhỏ, cần gradient accumulation lên 256

B `max_length=128` cắt mất chữ, cần tăng lên 512 token

C Thiếu warmup, thêm warmup là đủ, giữ learning rate cũ

D Learning rate quá lớn đối với trọng số pretrained

PHỎNG THEO NOAI 2026 Singapore · Chung kết · Câu 21

CÂU 6

Từ head-only, đội mở thêm `layer4` của ResNet-50, khoảng 15 triệu tham số, rồi train bằng Adam ở float32. Phần bộ nhớ tăng thêm cho gradient và trạng thái optimizer của những tham số vừa mở, chưa tính activation, là khoảng bao nhiêu MB, với 1 MB bằng 10^6 byte?

A 60

B 120

C 180

D 240

CÂU 7

BatchNorm có running mean cũ là 2, batch mean là 8, và `momentum=0.25` theo quy ước của PyTorch. Running mean mới bằng bao nhiêu?

A 3.5

B 5

C 6.5

D 8

CÂU 8

Hai checkpoint A và B có validation loss lần lượt là 0.30 và 0.35, accuracy là 0.88 và 0.90. Quy trình đã định từ trước là chọn theo loss nhỏ nhất. Chọn checkpoint nào, và báo accuracy nào?

A Chọn A, báo accuracy 0.90 của B

B Chọn A, báo accuracy 0.88

C Chọn B, báo accuracy 0.90

D Trộn trọng số của A và B, báo 0.90

CÂU 9

Học head, còn encoder được giữ cố định chỉ bằng cách không đưa vào optimizer; tham số của nó vẫn `requires_grad=True`. Muốn khỏi tốn bộ nhớ cho graph của encoder bằng `torch.no_grad()`. Đặt khối ấy ở đâu?

A Bọc cả dòng `loss = criterion(head(encoder(x)), y)` trong khối

B Chỉ bọc `head(z)`, còn `z = encoder(x)` tính bên ngoài khối

C Không cần khối, gọi `encoder.eval()` là đủ tiết kiệm

D Chỉ bọc `z = encoder(x)`, còn head và loss tính bên ngoài khối

CÂU 10

Head-only đạt accuracy 0.86 trên train và 0.84 trên validation. Mở toàn bộ encoder thì đạt 0.99 và 0.79, trên cùng cách chia. Bước tiếp theo hợp lý là gì?

A Chọn full fine-tuning, vì nó học được nhiều tham số hơn

B Mở thêm tham số, vì train accuracy vẫn chưa bằng 1

C Mở ít hơn hoặc giảm learning rate của encoder, rồi so lại với head-only

D Chọn full fine-tuning, nếu test công khai cho điểm cao hơn

Đọc thêm

- PyTorch — transfer learning tutorial: hai nhánh fixed feature extractor và fine-tuning cùng vòng train/validation.
- Dive into Deep Learning — fine-tuning: cách thay head và dùng bước cập nhật khác nhau cho trọng số cũ/mới.
- PyTorch — BatchNorm1d: running statistics, affine parameters và nghĩa của momentum.
- PyTorch — autograd mechanics: quan hệ giữa freeze, grad mode và eval mode.

- Transformers — sequence classification: cùng quy trình với model văn bản: tokenizer, head phân loại mới, learning rate nhỏ và chấm validation.
- Devlin và cộng sự — BERT, phụ lục A.3: dải learning rate, batch size và số epoch khi fine-tune.
- Howard, Ruder — ULMFiT: mở dần từng tầng và learning rate khác nhau theo tầng khi fine-tune model ngôn ngữ.

Nguồn ảnh: ảnh nhỏ lấy từ bộ CIFAR-10 (Krizhevsky, 2009).

Tải gói recipe của chương — gồm các module phụ và hướng dẫn chuẩn bị asset.